

TEMA 2. Encapsulación

1. En Programación Orientada a Objetos (POO), ¿Qué buscan la **encapsulación** y la **ocultación** de información? Enumera brevemente algunas ventajas de la ocultación de información.

La encapsulación en POO tiene como objetivo agrupar los datos (atributos) y los comportamientos (métodos) relacionados dentro de una unidad cohesiva llamada clase, estableciendo límites claros entre la interfaz y la implementación. Por su parte, la ocultación de información es un principio derivado que restringe el acceso directo a los detalles internos del objeto, permitiendo únicamente la interacción mediante una interfaz pública controlada. Esto garantiza que los mecanismos internos puedan modificarse sin afectar al código externo que utiliza la clase, siempre que se preserve dicha interfaz.

Entre las ventajas de la ocultación de información se encuentra una mayor mantenibilidad, ya que los cambios en la implementación no requieren ajustes en los módulos dependientes. También refuerza la integridad del estado del objeto, al evitar modificaciones arbitrarias que podrían violar condiciones lógicas esenciales. Además, favorece la modularidad y la reutilización del código, al permitir que cada clase sea desarrollada y probada de forma aislada, reduciendo la complejidad global del sistema.

2. ¿Qué se entiende por la **interfaz pública** de un objeto o clase en POO? Describe brevemente cómo se relaciona con la ocultación de información.

La interfaz pública de una clase está constituida por el conjunto de métodos y, en menor medida, atributos accesibles desde fuera de la clase, generalmente marcados con el modificador **public**. Esta interfaz define el contrato de uso: especifica qué operaciones puede realizar un objeto sin revelar cómo se implementan internamente. Representa la única vía legítima de interacción con el objeto para el código externo.

La ocultación de información se apoya directamente en esta interfaz pública para esconder los detalles de implementación. Al restringir el acceso a los componentes internos mediante modificadores como **private**, se logra que la lógica interna pueda evolucionar sin romper la compatibilidad con el código cliente. De este modo, la interfaz actúa como una barrera protectora que preserva la estabilidad del sistema frente a cambios internos.

3. Brevemente: ¿Por qué hay que ser conscientes y diseñar con cuidado la **interfaz pública** de una clase? ¿Es fácil cambiarla?

Diseñar con cuidado la interfaz pública es fundamental porque, una vez publicada y adoptada por otros módulos o desarrolladores, cualquier modificación puede generar incompatibilidades hacia atrás. Alterar la firma de un método, eliminar uno existente o cambiar su comportamiento observable obliga a actualizar todo el código que depende de él, lo que incrementa el riesgo de errores y el costo de mantenimiento en sistemas de mediana o gran escala.

Por ello, no es fácil cambiar una interfaz pública una vez establecida. Se recomienda planificarla desde el inicio con una perspectiva a largo plazo, exponiendo únicamente lo esencial y evitando comprometerse con detalles de implementación. Una interfaz minimalista y estable facilita la evolución del software, permitiendo refinar la lógica interna sin afectar a los consumidores de la clase.

4. ¿Qué son las **invariantes de clase** y por qué la ocultación de información nos ayuda?

Las invariantes de clase son condiciones lógicas que deben mantenerse verdaderas durante toda la vida útil de un objeto, independientemente de las operaciones que se realicen sobre él. Por ejemplo, en una clase que representa un ángulo, una invariante podría ser que el valor siempre esté comprendido entre 0 y 360 grados. Estas restricciones garantizan la coherencia interna del estado del objeto.

La ocultación de información contribuye a preservar estas invariantes al impedir el acceso directo a los atributos internos. Al obligar a que todas las modificaciones pasen por métodos controlados, es posible incluir validaciones que aseguren el cumplimiento de las condiciones establecidas. De este modo, se evita que el estado del objeto quede en una configuración inválida debido a asignaciones externas no supervisadas.

5. Pon un ejemplo de una clase **Punto** en **Java**, con dos coordenadas, **x** e **y**, de tipo **double**, con un método **calcularDistanciaAOrigen**, y que haga uso de la ocultación de información. ¿Cuál es la interfaz pública de la clase **Punto**? ¿Qué significa **public** y **private**?

```
public class Punto {  
    private double x;  
    private double y;  
  
    public Punto(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public double calcularDistanciaAOrigen() {  
        return Math.sqrt(x * x + y * y);  
    }  
}
```

La interfaz pública de la clase **Punto** está formada por el constructor y el método **calcularDistanciaAOrigen**, ambos accesibles desde cualquier otro código. Los atributos **x** e **y** permanecen ocultos gracias al modificador **private**, que restringe su acceso exclusivamente a los métodos definidos dentro de la propia clase.

El modificador **public** indica que un elemento es accesible desde cualquier parte del programa, mientras que **private** limita el acceso al ámbito de la clase en la que se declara. Esta distinción es fundamental para la encapsulación, ya que permite exponer únicamente lo necesario para el uso correcto del objeto, protegiendo al mismo tiempo su estado interno de manipulaciones indebidas.

6. En Java, ¿A quiénes se pueden aplicar los modificadores **public** o **private**?

En Java, los modificadores de acceso **public** y **private** pueden aplicarse a clases (aunque **private** solo es válido para clases anidadas), métodos, atributos y constructores. Para las clases de nivel superior, únicamente se permite **public** o el acceso por defecto (sin modificador), mientras que las clases internas sí admiten

private. Los métodos y atributos, por su parte, pueden declararse con cualquiera de los cuatro niveles de visibilidad disponibles en el lenguaje.

Esta flexibilidad permite un control granular sobre la exposición de los componentes del sistema. Por ejemplo, un atributo puede declararse **private** para ocultarlo, mientras que un método que lo manipula se marca como **public** para ofrecer una interfaz controlada. La elección adecuada de estos modificadores es esencial para lograr un diseño robusto y bien encapsulado.

7. En POO, la visibilidad puede ser pública o privada, pero ¿existen más tipos de visibilidad? ¿Qué ocurre en Java? ¿Y en otros lenguajes?

Además de pública y privada, Java incorpora dos niveles adicionales de visibilidad: **protected** y el acceso por defecto (también llamado package-private). El modificador **protected** permite el acceso dentro del mismo paquete y por parte de las subclases, incluso si están en paquetes distintos. El acceso por defecto, que se obtiene al omitir cualquier modificador, restringe la visibilidad al paquete actual.

Otros lenguajes ofrecen variantes adicionales; por ejemplo, C++ incluye la amistad (**friend**) para otorgar acceso selectivo a clases o funciones específicas, mientras que Python emplea convenciones de nombres (como prefijos con guion bajo) para sugerir privacidad, aunque sin restricciones estrictas en tiempo de compilación. Estos mecanismos reflejan distintos enfoques para equilibrar encapsulación y flexibilidad según las necesidades del diseño.

8. Responde: Los miembros de instancia privados de un objeto están ocultos para (a) otras clases o (b) otras instancias, aunque sean de la misma clase. Pon un ejemplo añadiendo un método **calcularDistanciaAPunto(Punto otro)** y explica la respuesta.

En Java, los miembros privados están ocultos para otras clases, pero no para otras instancias de la misma clase. Esto significa que un método definido dentro de una clase puede acceder a los campos privados de cualquier objeto de ese tipo, no solo del propio (**this**). La privacidad se aplica a nivel de clase, no de instancia individual.

```
public double calcularDistanciaAPunto(Punto otro) {  
    double dx = this.x - otro.x;  
    double dy = this.y - otro.y;  
    return Math.sqrt(dx * dx + dy * dy);  
}
```

En este ejemplo, el método accede tanto a **this.x** como a **otro.x**, ambos privados, sin generar error de compilación. Esto demuestra que la restricción de **private** no impide el acceso entre instancias de una misma clase, sino únicamente desde clases externas, preservando así la encapsulación a nivel de módulo.

9. ¿Qué son los métodos "getter" y "setter" en los lenguajes orientados a objetos?

Los métodos "getter" y "setter" son métodos públicos diseñados para acceder y modificar, respectivamente, los atributos privados de una clase. Un getter típicamente devuelve el valor del atributo sin parámetros,

mientras que un setter recibe un valor y lo asigna al atributo, a menudo incluyendo validaciones o lógica adicional antes de realizar la asignación.

Estos métodos refuerzan la encapsulación al proporcionar un punto de control centralizado para las operaciones de lectura y escritura. Permiten, por ejemplo, validar entradas, notificar cambios o calcular valores derivados sin exponer directamente el estado interno. Aunque pueden parecer redundantes cuando no incluyen lógica adicional, su uso facilita futuras evoluciones de la implementación sin alterar la interfaz pública.

10. Cuando nos referimos a que la ocultación de información mejora la "seguridad" del programa, ¿nos referimos a que no pueda ser "hackeado"?

No, en este contexto la "seguridad" no alude a la protección contra ataques maliciosos o intrusiones externas. Más bien, se refiere a la robustez y fiabilidad del software frente a errores de programación involuntarios. La ocultación evita que el estado interno de un objeto sea corrompido por usos incorrectos desde fuera de la clase, garantizando que las operaciones respeten las invariantes establecidas.

Esta forma de seguridad contribuye a la integridad del sistema al reducir la superficie de error. Al centralizar el acceso al estado mediante métodos controlados, se minimiza el riesgo de inconsistencias derivadas de modificaciones directas no validadas. Es, por tanto, una medida de calidad del código orientada a la prevención de fallos lógicos, no a la ciberseguridad per se.

11. ¿Qué diferencia hay entre **miembro de instancia** y **miembro de clase**? ¿Los miembros de clase también se pueden ocultar?

Los miembros de instancia pertenecen a cada objeto individual creado a partir de la clase; cada instancia mantiene su propia copia de dichos miembros. Por el contrario, los miembros de clase, declarados con el modificador **static**, son compartidos por todas las instancias y existen incluso sin crear objetos de la clase. Representan estado o comportamiento asociado a la clase en sí, no a objetos concretos.

Sí, los miembros de clase también pueden ocultarse mediante el modificador **private**. Esto es recomendable cuando el estado global de la clase debe protegerse de accesos o modificaciones externas no autorizadas. Por ejemplo, un contador estático que registra el número de instancias creadas debería ser privado y actualizarse exclusivamente dentro de los constructores o métodos controlados de la clase.

12. Brevemente: ¿Tiene sentido que los constructores sean privados?

Sí, tiene sentido en determinados patrones de diseño donde se desea controlar estrictamente la creación de instancias. Un constructor privado impide que otras clases instancien directamente la clase, forzando el uso de métodos de fábrica estáticos o de instancias predefinidas. Esto es habitual en el patrón singleton, donde solo debe existir una única instancia, o en clases de utilidad que no requieren ser instanciadas.

Esta práctica refuerza el encapsulamiento al centralizar toda la lógica de construcción en puntos específicos del código. Permite, además, validar condiciones previas a la creación del objeto o gestionar recursos de forma controlada, garantizando que todas las instancias se inicialicen de manera consistente y segura.

13. ¿Cómo se indican los **miembros de clase** en Java? Pon un ejemplo, en la clase **Punto** definida anteriormente, para que incluya miembros de

clase que permitan saber cuáles son los valores **x** e **y** máximos que se han establecido en todos los puntos que se hayan creado hasta el momento.

En Java, los miembros de clase se declaran utilizando el modificador **static**. A continuación se muestra una versión modificada de la clase **Punto** que incluye campos estáticos privados para rastrear los valores máximos de las coordenadas:

```
public class Punto {
    private double x;
    private double y;
    private static double maxX = Double.NEGATIVE_INFINITY;
    private static double maxY = Double.NEGATIVE_INFINITY;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
        if (x > maxX) maxX = x;
        if (y > maxY) maxY = y;
    }

    public static double obtenerMaxX() { return maxX; }
    public static double obtenerMaxY() { return maxY; }
}
```

Los campos **maxX** y **maxY** son estáticos y privados, lo que garantiza que solo los métodos de la propia clase puedan actualizarlos. Cada vez que se crea un nuevo punto, el constructor compara sus coordenadas con los máximos actuales y los actualiza si corresponde. Los métodos estáticos públicos **obtenerMaxX** y **obtenerMaxY** proporcionan acceso de solo lectura a estos valores globales.

14. Como sería un método factoría dentro de la clase **Punto** para construir un **Punto** a partir de dos coordenadas, pero que las redondee al entero más cercano. Escribe sólo el código del método, no toda la clase ¿Has usado **static**?

```
public static Punto crearPuntoRedondeado(double x, double y) {
    return new Punto(Math.round(x), Math.round(y));
}
```

Sí, se ha utilizado el modificador **static** para definir el método como miembro de clase. Esto permite invocarlo sin necesidad de una instancia previa de **Punto**, como en **Punto.crearPuntoRedondeado(3.7, 4.2)**. El uso de **static** es característico de los métodos de fábrica, ya que su propósito es crear y devolver nuevas instancias, no operar sobre un objeto existente.

15. Cambia la implementación de **Punto**. En vez de dos **double**, emplea un array interno de dos posiciones, intentando no modificar la interfaz

pública de la clase.

```
public class Punto {  
    private double[] coords = new double[2];  
  
    public Punto(double x, double y) {  
        coords[0] = x;  
        coords[1] = y;  
    }  
  
    public double calcularDistanciaAOrigen() {  
        return Math.sqrt(coords[0] * coords[0] + coords[1] * coords[1]);  
    }  
}
```

La interfaz pública permanece inalterada: el constructor sigue recibiendo dos parámetros `double` y el método `calcularDistanciaAOrigen` mantiene su firma y comportamiento observable. Los usuarios de la clase no perciben el cambio interno de dos campos independientes a un array, lo que demuestra cómo la encapsulación permite modificar la implementación sin afectar al código cliente.

Este enfoque refuerza el principio de ocultación, ya que los detalles sobre cómo se almacenan las coordenadas quedan completamente aislados. Futuras optimizaciones, como cambiar a coordenadas polares o añadir validaciones, podrían implementarse sin romper la compatibilidad con el uso existente de la clase.

16. Si un atributo va a tener un método "getter" y "setter" públicos, ¿no es mejor declararlo público? ¿Cuál es la convención más habitual sobre los atributos, que sean públicos o privados? ¿Tiene esto algo que ver con las "invariantes de clase"?

Aunque un atributo cuente con getter y setter públicos, no es recomendable declararlo público directamente. La convención habitual en Java y otros lenguajes orientados a objetos es mantener todos los atributos como `private`, incluso cuando existen métodos de acceso. Esto preserva la flexibilidad de añadir lógica de validación, notificación o transformación en el futuro sin alterar la interfaz pública.

Esta práctica está estrechamente relacionada con las invariantes de clase. Un setter permite garantizar que cualquier valor asignado cumpla condiciones específicas antes de modificar el estado interno. Si el atributo fuera público, cualquier código externo podría violar esas invariantes sin control, comprometiendo la integridad del objeto. La encapsulación, por tanto, no se trata solo de ocultar datos, sino de proteger su consistencia lógica.

17. ¿Qué significa que una clase sea **immutable**? ¿qué es un método modificador? ¿Un método modificador es siempre un "setter"? ¿Tiene ventajas que una clase sea immutable?

Una clase es immutable cuando sus instancias no pueden cambiar de estado después de su construcción; todos sus atributos son finales y no existen métodos que los modifiquen. Un método modificador es cualquier método que altera el estado interno del objeto, ya sea un setter tradicional o una operación más compleja como `agregarElemento` en una lista.

No todos los métodos modificadores son setters; un setter suele asignar directamente un valor a un atributo, mientras que otros modificadores pueden realizar cálculos o transformaciones antes de actualizar el estado. Las clases inmutables ofrecen ventajas significativas, como seguridad en entornos concurrentes (al no requerir sincronización), facilidad para razonar sobre el código y compatibilidad con estructuras como claves en mapas, al garantizar que el estado no variará tras su inserción.

18. ¿Es recomendable incluir métodos "setter" siempre y como convención?

No es recomendable incluir setters de forma sistemática. La decisión debe basarse en el diseño conceptual de la clase: si el estado debe permanecer constante tras la construcción, como en clases que representan valores (puntos, fechas, identificadores), se prefieren clases inmutables sin setters. Los setters deben añadirse únicamente cuando el modelo del dominio exija modificaciones posteriores al objeto.

La convención moderna favorece la inmutabilidad por defecto, añadiendo setters solo cuando son estrictamente necesarios. Esto refuerza las invariantes, reduce efectos secundarios no deseados y facilita el mantenimiento. La encapsulación no implica proporcionar acceso de escritura indiscriminado, sino controlar cuidadosamente cómo y cuándo puede modificarse el estado.

19. ¿La clase `String` en Java es mutable o inmutable? ¿Qué ocurre al concatenar dos cadenas? ¿Qué debemos hacer si vamos a hacer una operación que implique concatenar muchas veces para construir paso a paso una cadena muy larga?

La clase `String` en Java es inmutable; cualquier operación que parezca modificarla, como la concatenación con el operador `+`, en realidad crea un nuevo objeto `String` con el contenido resultante. Por ejemplo, `s1 + s2` genera una nueva cadena sin alterar `s1` ni `s2`, lo que implica una sobrecarga de memoria y procesamiento en operaciones repetidas.

Para concatenar eficientemente múltiples cadenas, especialmente en bucles, se recomienda utilizar `StringBuilder` (o `StringBuffer` en contextos concurrentes). Estas clases son mutables y permiten construir cadenas de forma incremental sin generar objetos intermedios innecesarios. Al finalizar, se invoca `toString()` para obtener el resultado como un `String` inmutable, optimizando así el rendimiento y el uso de memoria.

20. En POO ¿Cómo se comparan objetos de una misma clase? ¿Por su contenido o por su identidad? ¿Qué es el método `equals` en Java? ¿Qué hace por defecto? ¿Cómo se deben comparar dos cadenas en Java?

En POO, los objetos pueden compararse por identidad (si son la misma referencia en memoria) o por contenido (si sus estados son equivalentes según criterios lógicos). En Java, el operador `==` compara identidad, mientras que el método `equals` está diseñado para comparar contenido semántico.

Por defecto, `equals` heredado de la clase `Object` compara identidad, comportándose igual que `==`. Sin embargo, muchas clases estándar, como `String`, sobrescriben `equals` para comparar contenido. Para cadenas, siempre debe utilizarse `equals` en lugar de `==`, ya que dos cadenas con el mismo texto pueden ser objetos distintos en memoria, dando un resultado falso con el operador de igualdad referencial.

21. ¿Qué son las clases "wrapper" en un lenguaje de programación orientado a objetos? ¿Cómo se hace? ¿Es un proceso automático? ¿Qué ventajas tienen? ¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

Las clases wrapper son envoltorios que permiten tratar tipos primitivos como objetos. En Java, por ejemplo, `Integer` envuelve a `int`, `Double` a `double`, etc. Esto es necesario en contextos que requieren objetos, como colecciones genéricas o reflexión, donde los primitivos no son admitidos directamente.

Java proporciona autoboxing y unboxing, procesos automáticos realizados por el compilador que convierten entre primitivos y sus wrappers sin código explícito. Las ventajas incluyen interoperabilidad con APIs basadas en objetos y la posibilidad de almacenar valores nulos. No todos los lenguajes necesitan wrappers; lenguajes como Python o Ruby tratan todos los tipos como objetos desde el inicio, eliminando esta distinción artificial.

22. ¿En POO qué es un **tipo de dato enumerado**? ¿En Java, un tipo de dato enumerado es una clase? ¿Qué ventajas tienen en términos de encapsulación los enumerados en Java?

Un tipo enumerado define un conjunto fijo y nombrado de constantes que representan los únicos valores válidos para una variable de ese tipo. En Java, los enumerados son clases especiales que extienden implícitamente `java.lang.Enum`, pudiendo contener campos, métodos, constructores y lógica asociada a cada constante.

Esta naturaleza de clase ofrece ventajas significativas en encapsulación: cada constante puede encapsular estado y comportamiento específico, manteniendo los detalles internos privados. Por ejemplo, un enumerado `DíaSemana` puede incluir un campo privado para el número de día y métodos públicos para operaciones relacionadas, garantizando que solo los valores predefinidos sean válidos y que su comportamiento esté centralizado y protegido.

23. Crea un tipo enumerado en Java que se llame `Mes`, con doce posibles instancias y que además proporcione métodos para obtener cuántos días tiene ese mes, el ordinal de ese mes en el año (1-12), empleando atributos privados y constructores del tipo enumerado. Añade además cuatro métodos para devolver si ese mes tiene algunos días de invierno, primavera, verano u otoño, indicando con un booleano el hemisferio (norte o sur, parámetro `enHemisferioNorte`). Es decir:

```
esDePrimavera(boolean esHemisferioNorte), esDeVerano(boolean esHemisferioNorte), esDeOtoño(boolean esHemisferioNorte), esDeInvierno(boolean esHemisferioNorte)
```

```
public enum Mes {  
    ENERO(31, 1), FEBRERO(28, 2), MARZO(31, 3), ABRIL(30, 4),  
    MAYO(31, 5), JUNIO(30, 6), JULIO(31, 7), AGOSTO(31, 8),  
    SEPTIEMBRE(30, 9), OCTUBRE(31, 10), NOVIEMBRE(30, 11), DICIEMBRE(31, 12);  
  
    private final int dias;  
    private final int ordinal;
```



```
Mes(int dias, int ordinal) {
    this.dias = dias;
    this.ordinal = ordinal;
}

public int getDias() { return dias; }
public int getOrdinal() { return ordinal; }

public boolean esDePrimavera(boolean enHemisferioNorte) {
    return enHemisferioNorte ? (this == MARZO || this == ABRIL || this ==
MAYO)
                                : (this == SEPTIEMBRE || this == OCTUBRE || this
== NOVIEMBRE);
}

public boolean esDeVerano(boolean enHemisferioNorte) {
    return enHemisferioNorte ? (this == JUNIO || this == JULIO || this ==
AGOSTO)
                                : (this == DICIEMBRE || this == ENERO || this ==
FEBRERO);
}

public boolean esDeOtoño(boolean enHemisferioNorte) {
    return enHemisferioNorte ? (this == SEPTIEMBRE || this == OCTUBRE || this
== NOVIEMBRE)
                                : (this == MARZO || this == ABRIL || this ==
MAYO);
}

public boolean esDeInvierno(boolean enHemisferioNorte) {
    return enHemisferioNorte ? (this == DICIEMBRE || this == ENERO || this ==
FEBRERO)
                                : (this == JUNIO || this == JULIO || this ==
AGOSTO);
}
```